Deutsche Fassung: docs/Anleitung-Docker-Synology.pdf

DulyInspect as a single instance

Setup with Docker and in Synology Container Manager

This guide describes how to run all of DulyInspect, app and server, as **one** container: on a Linux server with Docker or on a Synology DiskStation. It covers all environment variables, the data folder, HTTPS, backups and updates.

The app is available in German and English and follows the browser language (switchable at the bottom of every page). This guide quotes the German labels, with the English label in brackets where they first appear.

As of	2 November 2026 · app 2.89.0
Repository	github.com/tommx0/dulyinspect
Image	dulyinspect
Licence	GNU AGPL-3.0, free software (file LICENSE in the repository)
Status	Pre-release: tested automatically on Intel/AMD (amd64, GitHub workflow) and ARM (arm64); Safari/iOS and real Synology devices still pending (section 10)

Contents

1. Overview
2. Requirements
3. Building and providing the image
4. Environment variables
5. Data folder and shares
6. Running with Docker (Linux server)
7. Synology Container Manager
8. Backups
9. Updates
10. Known limitations and error messages

1. Overview

The image contains everything DulyInspect needs: the app (browser interface), the database (PostgreSQL), sign-in, file storage, real-time sync and the server functions. To the outside there is only **one port (8080)** and **one data folder (`/data`)**.

- **One instance per container:** one company, one database, one admin area. There are no tenants as in the Coolify stack.
- **The first admin** is created in the browser on the first visit, protected by a setup key from the data folder (optionally from environment variables for automated installs).
- **Updates** only replace the container. All data stays in the data folder, and the container backs up the database automatically before every version change.
- **Licence:** free software under the GNU AGPL-3.0. Use is free of charge, including in companies. Anyone who distributes a modified version or offers it as a service must publish their source code. In the app, „Lizenz“ (licence) at the bottom of every page leads to the licence text.
- **HTTPS** is provided by a reverse proxy in front of the container; on a Synology, the built-in one. Without HTTPS the camera (QR scanning) and installing the app on a phone do not work.

2. Requirements

Item	Requirement
Processor	Intel/AMD (`x86_64` , image for `linux/amd64`) or 64-bit ARM (`aarch64` , image for `linux/arm64`). The image must match the platform. Check via SSH with `uname -m` ; on a Synology also under **Info Center** → **General**.
Memory	Measured at around 500–600 MB in operation. Recommended: at least 1.5 GB free, i.e. a device with 2 GB of RAM or more (4 GB is better).

Disk space	Image around 1.9 GB; the transfer file is around 0.35 GB (GitHub download as ZIP; the unpacked `.tar` is around 1.2 GB). On top of that the data: database under 100 MB at first, photos and reports depending on use. Plus the backups.
Docker	Linux: Docker Engine 24 or later with the Compose plugin. Synology: DSM 7.2 or later with the **Container Manager** package.
Internet	On the **first start** and the first time user management is opened, the container downloads program libraries. It then needs outbound internet access (section 10).
Address and certificate	A name such as `inspection.example.com` that points to the server, and a certificate (Let's Encrypt), for use on site and on the road.

3. Building and providing the image

The image is handed over as a file (`.tar`). There are two ways to create it.

3.1 Via GitHub (recommended for Intel/AMD)

1. In the repository `tommx0/dulyinspect` on GitHub: **Actions** → **Build image (amd64)** → **Run workflow**.
2. The run takes about 10–15 minutes. It builds the image, starts it as a trial and checks first start, sign-in, file storage, real-time connection, backup, update and data retention. The file is only produced if everything passes.
3. On the run's page, under **Artifacts**, download `dulyinspect-<version>-amd64`. This is a ZIP file. Unpacking it gives the `.tar` file for the import (section 6 or 7.3). The file is available for download for 5 days.

With „Also publish to ghcr.io“ the image is also pushed to the private GitHub registry. This permanently uses storage of the GitHub account (around 0.35 GB per version).

3.2 Locally with build.sh

The script `build.sh` builds the image from the app source in the repository's `app/` folder and saves it as a `.tar` file.

```
git clone https://github.com/tommx0/dulyinspect.git
cd dulyinspect
./build.sh linux/arm64      # for ARM
./build.sh linux/amd64      # for Intel/AMD (on a Mac with Apple silicon see below)
test/smoke-test.sh dulyinspect:latest   # optional: the same checks as on GitHub
```

Result: `out/dulyinspect-<version>-<platform>.tar`, tagged `dulyinspect:<version>` and `dulyinspect:latest`. The app version included is listed in `app/SOURCE`.

> **Building for Intel/AMD on a Mac with Apple silicon:** Docker Desktop must be able to run x86 programs: turn on **Docker Desktop** → **Settings** → **General** → **“Use Rosetta for x86_64/amd64 emulation on Apple Silicon”** and restart Docker. Without this setting the build stops with `exec format error`.

On a Linux server with Docker, the image can also be built directly on the server (same commands, choose the matching platform). No file transfer is needed then.

4. Environment variables

Variable	Required	Meaning
`APP_URL`	yes	The address at which the app is opened in the browser, exactly as typed, with `https://` and without a path, e.g. `https://inspection.example.com`. It is used for the app's connection to the server, for sign-in and for the QR codes on labels and reports. Default: `http://localhost:8080` (for testing only).
`ADMIN_EMAIL`	no	Login name (email address) of the first admin. Default: `admin@lokal`.
`ADMIN_PASSWORD`	no	For automated installs only: creates the first admin at startup; at least 10 characters. Without this variable you create the admin in the browser (section 7.7).
`ADMIN_NAME`	no	Display name of the admin. Default: `Administrator`.
`ADMIN_CHANGE_PASSWORD`	no	`yes` (default): the admin must choose their own password at first sign-in. `no`: the start password stays valid.
`ADMIN_TWO_FACTOR`	no	`yes`: the admin must set up an authenticator app at first sign-in. Default: `no` (can be switched on later in the app under „Benutzer“ (users)).
`SETUP_WITHOUT_KEY`	no	`yes`: the initial setup in the browser does not ask for the setup key. **For testing only** on your own computer: whoever opens the page first becomes admin.
`MAX_ACTIVE_USERS`	no	Maximum number of users active at the same time (per licence). Default: `2`.

The former German names (`APP_ADRESSE` , `ADMIN_PASSWORT` , `ADMIN_PASSWORT_AENDERN` , `ADMIN_ZWEI_FAKTOR` , `MAX_AKTIVE_BENUTZER` , `EINRICHTUNG_OHNE_SCHLUESSEL`) and the values `ja` / `nein` still work as aliases.

> **The first admin is created exactly once:** on the very first start, while no account exists yet. After that the `ADMIN_*` variables have no effect. Changing them later does not reset a password. Remove `ADMIN_PASSWORD` from the configuration after the first start, because container settings can be read by every admin of the server or the Synology.

The normal case: without `ADMIN_PASSWORD` the container does not create a user. On the first visit the app shows the **Ersteinrichtung** (initial setup) form: there you create the first admin and print the encryption key sheet. So that no stranger gets there before you, the form asks for the **Einrichtungsschlüssel** (setup key). It is in the file `SETUP_KEY.txt` in the data folder (on a Synology in File Station under `docker/dulyinspect/data`) and in the container log. As soon as the admin has been created, the container deletes the file. After that, the app's **Inbetriebnahme** (commissioning wizard) guides you through company details, installations to inspect, checklists, legal notice and privacy policy, appearance, labels and users.

Changing the address: adjust `APP_URL` and recreate the container. The app picks up the new address the next time it loads. QR labels that have already been printed still contain the old address, though. The address should therefore be settled before the first labels are printed.

All other values (keys, database passwords) are generated by the container on first start and stored in the data folder. You do not set them.

5. Data folder and shares

Everything that must persist lives in the container under `/data`. This path is mapped to a folder on the server or the Synology (called `data` below). The container creates the subfolders itself on first start and sets the permissions. An existing data folder with the former German names (`geheim/`, `sicherung/` etc.) is renamed automatically on the next start.

Subfolder	Contents
`postgres/`	Database. Never copy or open it while the container is running (section 8).
`storage/`	Photos, signatures, inspection reports (PDF), certificates and evidence, logo.
`storage-xattr/`	Metadata about the files (type, cache). Only used if the folder does not support extended attributes, e.g. with Docker Desktop or an SMB/NFS share. Part of the data; do not delete.
`secrets/`	Keys and passwords of the instance (`instance.env`). **Never delete:** without them all sign-ins become invalid, and encrypted entries in the database can no longer be read. Include in every backup and keep confidential.
`backups/`	Database backups: automatically before every version change (`pre-*.sql.gz`, the last 5) and the scheduled ones (`live-*.sql.gz`, section 8).
`caddy/`, `version`, `.lock`	Internal housekeeping (web server, last set-up version, lock).

> **A data folder belongs to exactly one container.** A second container on the same folder would corrupt the database. The container prevents this itself: it exits with `ERROR: /data is already in use by another container. Stop the other one first.` So when updating, always stop the old container first.

Inside the folder the files belong to various internal users of the container. In the server's file view they therefore show unfamiliar owners or appear unreadable. That is correct. Do not change owners or permissions from outside.

6. Running with Docker (Linux server)

> **Docker Desktop (Mac, Windows):** use a named Docker volume instead of a folder from the computer (`dulyinspect-data:/data` plus a `volumes:` section, see README). Docker Desktop cannot keep Linux file owners on a host folder, and Postgres then refuses to start (`ERROR: /data cannot keep file owners …`). Linux servers and Synology shared folders are not affected.

1. Create a folder, e.g. `/opt/dulyinspect`, and copy the file `docker-compose.yml` from the repository into it.
2. Load the image (not needed if it was built on the server):

   ```
   docker load -i dulyinspect-2.89.0-amd64.tar
   ```

3. Enter the environment variables in `docker-compose.yml` (section 4), above all `APP_URL`.
4. Start and follow the log:

   ```
   cd /opt/dulyinspect
   docker compose up -d
   docker compose logs -f
   ```

   The container is ready once the log shows `Setup complete, app 2.89.0 at https://….` On first start this takes about 10–30 seconds.
5. Put a reverse proxy with HTTPS in front (Caddy, nginx, Traefik …) that forwards to `http://127.0.0.1:8090`. It must pass **WebSocket** through (headers `Upgrade` and `Connection`), otherwise real-time sync between several inspectors does not work. Port 8090 should only be reachable locally, i.e. `"127.0.0.1:8090:8080"` in `ports` if the proxy runs on the same server.
6. Open the address in the browser and create the first admin (section 7.7). `cat data/SETUP_KEY.txt` shows the setup key.

The compose file from the repository:

```
services:
  dulyinspect:
    image: dulyinspect:latest
    container_name: dulyinspect
    restart: unless-stopped
    ports:
      - "8090:8080"            # left: port on the server/NAS, right: always 8080
    volumes:
      - ./data:/data           # all data; never delete on updates
    environment:
      APP_URL: "https://inspection.example.com"   # address the app is opened at in the
browser
      MAX_ACTIVE_USERS: "2"
      # The first admin is created in the browser on first visit (setup key:
data/SETUP_KEY.txt).
      # Only for automated installs: create the admin at startup (first start only)
      # ADMIN_EMAIL: "admin@lokal"
      # ADMIN_NAME: "Administrator"
      # ADMIN_PASSWORD: "at-least-10-characters"
    stop_grace_period: 60s
```

Without Compose, the same works with a single command:

```
docker run -d --name dulyinspect --restart unless-stopped --stop-timeout 60 \
  -p 8090:8080 -v /opt/dulyinspect/data:/data \
  -e APP_URL=https://inspection.example.com \
  dulyinspect:latest
```

Useful commands: `docker compose stop` (clean stop), `docker compose start`, `docker compose logs --tail 100`. Do not stop the container with `docker kill` or `docker rm -f`. It survives that, but then needs a database recovery on the next start.

7. Synology Container Manager

The labels follow DSM 7.2 in English. Menu items may be named slightly differently in other versions.

7.1 Install Container Manager and check the platform

1. **Package Center** → install **Container Manager**. This creates the shared folder `docker`.
2. **Info Center** → **General**: read the processor. Intel, AMD or Celeron means image `amd64`; an ARM processor (e.g. Realtek RTD1619B) means `arm64`.

7.2 Set up the share and folders

1. In **File Station**, inside the folder `docker`, create the subfolder `dulyinspect` and in it the folder `data`. Result: `/docker/dulyinspect/data` (on the volume: `/volume1/docker/dulyinspect/data`).
2. **Control Panel** → **Shared Folder** → **docker** → **Edit** → **Permissions**: access for administrators only. The folder contains the instance keys in `data/secrets`, and the compose file may contain the start password.
3. Do **not** share `docker` with ordinary users via SMB or AFP, and do not change anything by hand in the `data` folder.
4. For backups (section 8) a Btrfs volume is recommended. It also makes snapshots of the share possible.

7.3 Import the image

1. Copy the file `dulyinspect-<version>-amd64.tar` (or `-arm64`) into the `docker` share, e.g. with File Station (Upload).
2. **Container Manager** → **Image** → **Add** → **Add From File** and select the file. The import takes a few minutes.
3. Afterwards `dulyinspect` appears in the list with the tags `latest` and the version number.

Alternatively via SSH (**Control Panel** → **Terminal & SNMP** → **Enable SSH service**): `sudo docker load -i /volume1/docker/dulyinspect-2.89.0-amd64.tar`

7.4 Create a project (recommended)

A project is a compose file managed by Container Manager. That way all settings are in one place, and updates are easier.

1. **Container Manager** → **Project** → **Create**.
2. **Project name:** `dulyinspect` · **Path:** `/docker/dulyinspect` · **Source:** “Create docker-compose.yml”.
3. Paste the compose file from section 6 and adjust the values:
   - `APP_URL`: the future HTTPS address (section 7.6), e.g. `https://inspection.example.com`.
   - Port to the left of `:8080`: a free port on the Synology, e.g. `8090`. Ports 5000/5001 are used by DSM itself.

- Leave `./data:/data` as it is: `./` refers to the project folder `/docker/dulyinspect`.

4. Skip the page for the web portal settings (Web Station); do not set anything up there.
5. **Done** with “Start the project once it is created”. Container Manager creates and starts the container.
6. **Container** → **dulyinspect** → **Log**: wait for `Setup complete, app 2.89.0 at ….`

7.5 Alternative: container without a project

If you prefer clicking: **Image** → **dulyinspect:latest** → **Run**.

- **General Settings:** container name `dulyinspect`, turn on “Enable auto-restart”.
- **Port Settings:** local port `8090` → container port `8080`, TCP.
- **Volume Settings:** “Add Folder” → `docker/dulyinspect/data` → mount path `/data`, not read-only.
- **Environment:** add the variables from section 4 one by one.
- **Network:** `bridge` (default).

7.6 HTTPS via the Synology reverse proxy

1. Point the name (e.g. `inspection.example.com`) to the Synology in DNS. For access from outside, the router forwards port 443 to the Synology. For use on site only, an entry in the internal DNS is enough.
2. **Control Panel** → **Login Portal** → **Advanced** → **Reverse Proxy** → **Create**:

	Source	Destination
Protocol	HTTPS	HTTP
Hostname	`inspection.example.com`	`localhost`
Port	443	8090

   Under Source, turn on “Enable HSTS”.
3. In the same entry: **Custom Header** → **Create** → **WebSocket**. This adds `Upgrade` and `Connection`. **Without this step** inspectors do not see each other's changes immediately, only on the next reload.
4. **Control Panel** → **Security** → **Certificate** → **Add** → “Add a new certificate” → “Get a certificate from Let's Encrypt” for `inspection.example.com`. Then, under **Settings**, assign the certificate to the reverse proxy entry. Let's Encrypt needs port 80 from outside for this.
5. Do not expose port 8090 to the internet, i.e. do not forward it in the router. The Synology firewall can additionally restrict it to `localhost`.

7.7 Initial setup and commissioning

1. In File Station, open `docker/dulyinspect/data/SETUP_KEY.txt` and copy the key. It is also in the log: **Container Manager** → **Container** → **dulyinspect** → **Log**, in the line `Initial setup in the browser at …, setup key: XXXX-XXXX-…`.
2. Open `https://inspection.example.com`. In the **Ersteinrichtung** (initial setup) form, enter the key, your name, your email address (future login name) and a password.
3. Download the encryption **Schlüsselblatt** (key sheet), print it and keep it safe: without it, encrypted data cannot be recovered.

4. After that, the **Inbetriebnahme** (commissioning wizard) guides you step by step through the setup. Every step can be skipped and done later under **Einstellungen** → **Inbetriebnahme** (settings → commissioning).
5. On the phone, open the address and install it via “Add to Home Screen”. Only then does offline use work reliably, especially on the iPhone.

8. Backups

The database files under `data/postgres` must **not** be copied while the container is running. Such a copy may be unusable. These three methods are safe, ideally combined:

8.1 Daily database backup (Task Scheduler)

The container comes with a backup script. While running, it writes a complete, consistent backup of the database to `data/backups/live-<time>.sql.gz` and keeps the last 14 (adjustable).

1. **Control Panel** → **Task Scheduler** → **Create** → **Scheduled Task** → **User-defined script**.
2. **General:** task “DulyInspect backup”, user `root`.
3. **Schedule:** daily, e.g. 02:30.
4. **Task Settings** → **User-defined script:**

   ```
   docker exec dulyinspect /usr/local/lib/instance/backup.sh 14
   ```

   Optionally turn on “Send run details by email” and “Send run details only when the script terminates abnormally”; the Synology then reports failed backups.

On a Linux server, do the same with cron, e.g. `30 2 * * * docker exec dulyinspect /usr/local/lib/instance/backup.sh 14`.

8.2 Backup to another device (Hyper Backup)

With **Hyper Backup** (or `rsync` / `restic` on Linux), back up these folders regularly, e.g. daily after the database backup, to another device or to the cloud:

- `docker/dulyinspect/data/backups`: the database backups
- `docker/dulyinspect/data/storage`: photos, reports, evidence
- `docker/dulyinspect/data/secrets`: the instance keys
- `docker/dulyinspect/docker-compose.yml`: configuration

The backups contain personal data and the instance keys. Therefore encrypt the backup destination (in Hyper Backup: “Client-side encryption”).

8.3 Snapshots (Btrfs)

With **Snapshot Replication**, the `docker` share can be captured as a snapshot, e.g. every hour. A snapshot captures the data folder at a single moment. When restored, the database behaves as after a power failure and repairs itself on start. Snapshots are handy for going back quickly, but do not replace a backup to another device.

8.4 Restoring

1. Stop the container (**Project** → **Action** → **Stop** or `docker compose stop`).
2. Restore the complete `data` folder from a snapshot or from a backup that was taken with the container **stopped** or as a snapshot.
3. Start the container. It checks the database state and version itself.

> Restoring an instance from the `.sql.gz` files alone (e.g. on new hardware without a snapshot) is not yet described as a finished procedure and has not been tested. Until then, keep snapshots or a copy of the whole data folder taken with the container stopped as the basis, and the database backups in addition.

9. Updates

An update only replaces the image. On the first start with a new app version, the following happens automatically:

1. Backup of the database to `data/backups/pre-<version>-<timestamp>.sql.gz`. If this fails, the container does not start.
2. Update of the database to the new version (one transaction per step; on an error the previous state is kept).
3. Start of the app with the new version. Devices fetch it without asking: immediately the next time the app is opened, otherwise on the next page change or when the app goes to the background. **Never during an inspection:** as long as an inspection is open on a device or an inspection page is displayed, the old version stays until the inspection is completed.

Synology

1. Build a new image file (section 3) and import it (section 7.3). It again carries the tag `latest`.
2. **Project** → **dulyinspect** → **Action** → **Stop**, then **Action** → **Build**. The container is recreated from the new image. Alternatively via SSH: `cd /volume1/docker/dulyinspect && sudo docker compose up -d --force-recreate`.
3. In the log, look for `Update 2.86.0 → …: backup to …` and `Setup complete`.

Docker

```
docker load -i dulyinspect-<new version>-amd64.tar   # or build again
cd /opt/dulyinspect
docker compose up -d          # detects the new image and recreates the container
docker compose logs -f
```

The container refuses to **go back to an older version** once the database has been updated (`… (older image?)`). In that case use the image of the new version again, or restore the backup from before the update (section 8.4).

Old images take up around 2 GB each. Images no longer in use can be deleted under **Container Manager** → **Image** (or with `docker image prune`). Keep the previous version for a few days to be safe.

10. Known limitations and error messages

Status of this pre-release

- **Tested** automatically on Intel/AMD (amd64, with every build on GitHub) and ARM (arm64): first start, sign-in, photos/files, real-time connection, backup, update with backup, data retention, locks. **Not yet** on a real Synology device: the menu items in section 7 follow DSM 7.2 but have not been clicked through on a device.
- **Safari/iPhone:** real-time sync has not yet been checked in Safari; Chrome has been checked.
- **No demo mode, no preview version:** the image is built for production use. There is no „Vorschauversion“ (preview version) notice or watermark, and demo mode is permanently off. New versions load without asking (section 9).
- **Internet access:** the server functions download libraries from the internet (npm) on first use. Without outbound access, creating the first admin from the variables and user management fail.
- **Postgres major version:** if a future image changes the database's major version (e.g. 15 to 17), the container does not start with old data (`ERROR: The data was created by Postgres …`). A separate guide will follow for this case; the data stays unchanged.

Log messages

Message	Meaning and remedy
`ERROR: /data is already in use by another container. Stop the other one first.`	A second container is using the same data folder. Stop the other one, then start again.
`ERROR: The database has schema …, this image only knows up to … (older image?)`	The image is older than the data. Use the newer image.
`ERROR: The data was created by Postgres …, this image has Postgres …`	Change of the database major version, see above. Nothing has been changed.
`Backup failed – start aborted.`	Usually not enough disk space. Free up space and start again.
`ERROR in migration … – data unchanged, backup in /data/backups.`	The database update failed; the previous state is kept. Send the log to support.
`ERROR creating the admin: …`	Usually a password that is too short (under 10 characters) or no internet access. Correct the variable and restart.
`Initial setup in the browser at …, setup key: XXXX-XXXX-… (also in data/SETUP_KEY.txt)`	Not an error: create the first admin in the browser with this setup key (section 7.7).
`First admin … created.`	Not an error: the first admin was created from `ADMIN_PASSWORD`. Now remove the variable from the configuration.

Common problems

Problem	Cause and remedy
Camera or QR scanning does not work, the app cannot be installed	The app is running without HTTPS. Open it via the reverse proxy with a certificate (section 7.6).
Changes by other inspectors only appear after a reload	The WebSocket headers are missing in the reverse proxy (section 7.6, step 3).
Sign-in works, then the error „keine Verbindung“ (no connection)	`APP_URL` does not match the address in the browser. Adjust it and recreate the container.
Container does not start, port in use	Change the left port (8090) to a free one, in the reverse proxy as well.